E^2-LLM: Efficient and Extreme Length Extension of Large Language Models

Jiaheng Liu*[1], Zhiqi Bai*[1], Yuanxing Zhang[1], Chenchen Zhang[1], Yu Zhang[1], Ge Zhang[2], Jiakai Wang[1], Haoran Que[1], Yukang Chen[3], Wenbo Su[1], Tiezheng Ge[1], Jie Fu[4], Wenhu Chen[2], Bo Zheng[1]

[1]Alibaba Group, [2]University of Waterloo, [3]The Chinese University of Hong Kong, [4]The Hong Kong University of Science and Technology

{ljh411989, baizhiqi.bzq}@taobao.com

arXiv:2401.06951v3 [cs.CL] 22 Feb 2024

Abstract

Typically, training LLMs with long context sizes is computationally expensive, requiring extensive training hours and GPU resources. Existing long-context extension methods usually need additional training procedures to support corresponding long-context windows, where the long-context training data (*e.g.*, 32k) is needed, and high GPU training costs are assumed. To address the aforementioned issues, we propose an Efficient and Extreme length extension method for Large Language Models, called E^2-LLM, with only one training procedure and dramatically reduced computation cost, which also removes the need to collect long-context data. Concretely, first, the training data of our E^2-LLM only requires a short length (*e.g.*, 4k), which reduces the tuning cost greatly. Second, the training procedure on the short training context window is performed only one time, and we can support different evaluation context windows at inference. Third, in E^2-LLM, based on RoPE position embeddings, we introduce two different augmentation methods on the scale and position index parameters for different samples in training. It aims to make the model more robust to the different relative differences when directly interpolating the arbitrary context length at inference. Comprehensive experimental results on multiple benchmark datasets demonstrate the effectiveness of our E^2-LLM on challenging long-context tasks.

1 Introduction

Large language models (LLMs) usually have a predefined context window length. For example, inputs to LLaMA models (Touvron et al., 2023a,b) must be fewer than 2,048 or 4096 tokens. This preset context window limit is frequently exceeded in applications such as long conversations, document summarization, or long-term reasoning (Zheng et al., 2023; Chen et al., 2023a). For these applications, LLMs with longer context windows are preferred. However, training an LLM from scratch with long context windows requires significant training costs. To address this problem, many long-context extension methods (Peng et al., 2023; Chen et al., 2023b) have been proposed to extend the context window of an existing pre-trained LLM.

One straightforward approach is called direct extrapolation, which fine-tunes an existing pre-trained LLM with a longer context window, However, the authors of Position Interpolation (PI) (Chen et al., 2023a) observe that models trained by direct extrapolation adapt to long context windows very slowly and direct extrapolation is inefficient in extending substantially longer context windows.

As shown in Fig. 1 (a), existing long-context extension methods (*e.g.*, PI) usually need additional training procedures to support corresponding longer-context windows, where the long-context training data is needed to collect, and training with high GPU memory usage is needed for each context window.

To address the aforementioned issues, as shown in Fig. 1 (b), we propose an Efficient and Extreme length extension method of LLMs, called **E^2-LLM**, to support the extreme length extension of LLMs with only one training procedure on short-context data and limited computation cost. Based on E^2-LLM, we can support different evaluation context windows well at inference by only changing one hyperparameter of RoPE (Su et al., 2021) according to the input context length. Specifically, first, in E^2-LLM, the training data only requires the commonly-used data with short lengths (*e.g.*, 4k), and we only need to train the LLM once and support interpolating the arbitrary context length at inference, which reduces the GPU usage costs largely. Second, in

* First two authors contributed equally.

Note that we follow (Chen et al., 2023b) to report the GPU memory by fine-tuning LLaMA2 7B on various context lengths with FlashAttention-2 (Dao, 2023) and DeepSpeed stage 2 (Rasley et al., 2020).

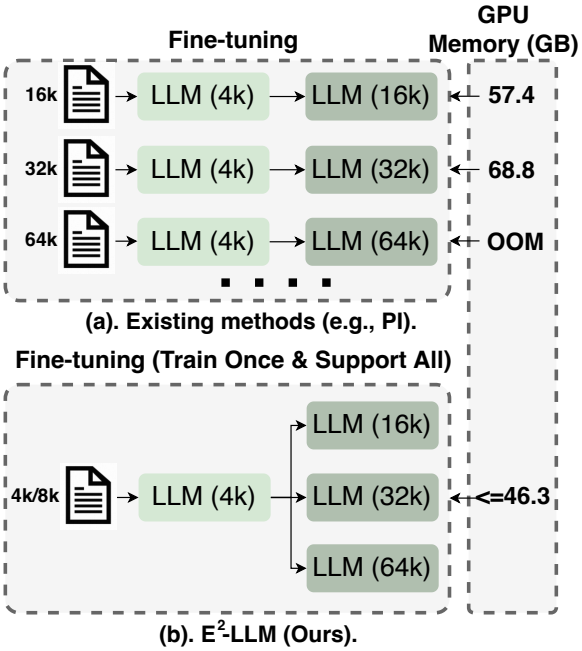

Figure 1: Comparison of existing long-context extension methods (*e.g.*, PI (Chen et al., 2023a)) and our E^2-LLM. "LLM (4k)" with light color means the LLM with default context window (*e.g.*, LLaMa2 with 4k). "LLM (16k/32k/64k)" with deep color means the LLM with extended context windows (16k/32k/64k) after fine-tuning. (a) For existing methods, we need to collect long-context data (*e.g.*, 16k/32k) and fine-tune the LLM models for different context extension windows with high GPU memory usage. (b). For E^2-LLM, we directly use the short-context data (*e.g.*, 4k/8k) by training LLMs only once time with acceptable GPU memory usage and support different evaluation context windows (*e.g.*, 16k/32k/64k) at inference.

our E^2-LLM, we first propose the augmentation on the scale parameter of PI from a predefined distribution (*e.g.*, uniform distribution), which aims to cover different position densities in training. Besides, we observe that only changing the scale parameter will make LLMs focus on a small range of absolute position indices. Therefore, in our E^2-LLM, to improve the generalization ability of our E^2-LLM, we further propose the augmentation on the position index parameters by introducing the position offsets on the absolute position indices of RoPE.

The contributions of our E^2-LLM are as follows:

- In our work, we first investigate the issues (*e.g.*, the large fine-tuning costs with long context data) of existing long-context extension methods, and propose the Efficient and Extreme length extension method (i.e., **E^2-LLM**) to train the LLMs once on the short-context data with limited GPU memory costs and support different evaluation context windows.
- In E^2-LLM, based on RoPE, we propose two augmentation strategies on the scale and position index parameters for different samples in training, which aims to support different evaluation context windows within one training procedure and improve the long-context abilities of LLMs.
- Comprehensive experimental results on multiple long-context benchmark datasets demonstrate the effectiveness and efficiency of our E^2-LLM method.

2 Related Works

Long-context Transformers. Extensive studies have aimed to increase transformers' ability to process longer text sequences. Strategies like using retrieval-based models (Karpukhin et al., 2020; Izacard et al., 2022) have been employed, which integrate additional documents and search findings into the context. Various efforts have adapted the multi-head attention by devising estimated alternatives (Wang et al., 2020; Beltagy et al., 2020; Kitaev et al., 2020; Bulatov et al., 2022; Ding et al., 2023) to mitigate the self-attention's inherently high computational demands. For example, Longformer (Beltagy et al., 2020) and BigBird (Zaheer et al., 2020) use a form of diluted attention for more extensive text. Meanwhile, other initiatives (Wu et al., 2022; Bulatov et al., 2022) have introduced memory-based systems to condense previous inputs and recall pertinent components. However, these approaches tend to fall short of the effectiveness of complete attention, thereby hindering the refinement of large pre-trained language models (LLMs) (Wu et al., 2024; Guo et al., 2023; Wang et al., 2023; Bai et al., 2024; Chai et al., 2024). Our approach differs in that it approximates the attention mechanism in a way that remains closely aligned with the conventional attention method, showing only a minimal discrepancy.

Long-context LLMs. Large language models (LLMs) such as LLaMA (Touvron et al., 2023a) and LLaMA2 (Touvron et al., 2023b) are originally trained with fixed context sizes, typically 2048 and 4096 tokens, respectively. Nonetheless, the cost of training LLMs with extended contexts from the

ground up is usually beyond the reach of the average research team. Consequently, recent studies have explored ways to expand the context length of these models during the fine-tuning stage. For example, Position Interpolation (Chen et al., 2023a) adapts the rotary position encoding technique (Su et al., 2021), which allows LLaMA to process contexts as long as 32768 tokens. Another method, Landmark attention (Mohtashami and Jaggi, 2023), achieves efficiency but at the cost of some accuracy, by compressing extended contexts into a set of retrieved tokens. In contrast, our strategy minimizes the expenses related to fine-tuning without compromising the efficacy of the original attention. It ensures that the model has complete and unchanged attention over the full input during the inference process. Other approaches, like ALiBi (Press et al., 2022), have been designed to train Transformers on shorter sequences and then apply them to longer ones at inference time, effectively extrapolating context length. However, these techniques are not as effective for pre-trained LLMs that rely on positional encodings with poor extrapolation capabilities, such as RoPE (Su et al., 2021). To overcome this, recent research has been directed towards modifying the positional embeddings of LLMs to handle longer texts. This includes methods like Position Interpolation (Chen et al., 2023a), NTK-aware position embeddings (ntk, 2023), and Yarn (Peng et al., 2023).

3 Background

3.1 Rotary Position Embedding (RoPE)

Transformer models require explicit positional information to be injected, where the positional encodings are used to represent the order of inputs. In this section, we take Rotary Position Embedding (RoPE) (Su et al., 2021) as an example, which is widely-used in many LLaMA-style models (Touvron et al., 2023a). In RoPE, given a position index $m \in [0, L)$ and an embedding vector $\mathbf{x} := [x_0, x_1, \ldots, x_{d-1}]^\top$, where L is the context window and d is the dimension of the attention head, a vector-valued complex function $\mathbf{f}(\mathbf{x}, m)$ is defined as follows:

$$\mathbf{f}(\mathbf{x}, m) = [(x_0 + \mathrm{i}x_1)e^{\mathrm{i}m\theta_0}, \ldots, (x_{d-2} + \mathrm{i}x_{d-1})e^{\mathrm{i}m\theta_{d/2-1}}]^\top, \tag{1}$$

where $\mathrm{i} := \sqrt{-1}$ is the imaginary unit and $\theta_j = 10000^{-2j/d}$. Based on RoPE, the self-attention score a is computed as follows:

$$\begin{aligned} a(m, n) &= \mathrm{Re}\langle \mathbf{f}(\mathbf{q}, m), \mathbf{f}(\mathbf{k}, n) \rangle \\ &=: a(m - n), \end{aligned} \tag{2}$$

where $\mathbf{q}$ and $\mathbf{k}$ are the query and key vectors for a specific attention head, respectively, and the detailed derivation is omitted. In Eq. 2, we observe that $a(m, n)$ is only dependent on relative position $m - n$ through trigonometric functions. Besides, RoPE is performed on both query and key embeddings for calculating the attention scores at each layer.

3.2 Position Interpolation

While the attention score in Eq. 2 only depends on the relative positions, its extrapolation performance is not great. Specifically, when direct extrapolation to larger unseen context windows in training, the perplexity will increase to very high numbers (i.e., $> 10^3$).

Recently, Position Interpolation (PI) (Chen et al., 2023a) has been proposed, where s is defined as the positional span between a query and a key, and L is defined as the size of the trained context window. Instead of direct extrapolation on the attention score to $s > L$, the attention score is defined as $\tilde{a}(s) = a(Ls/L')$, where L' is the extended longer context window. Formally, in PI, RoPE $\mathbf{f}$ is replaced by $\mathbf{f}'$ as follows:

$$\mathbf{f}'(\mathbf{x}, m) = \mathbf{f}\left(\mathbf{x}, \frac{mL}{L'}\right), \tag{3}$$

where position indices from $[0, L')$ to $[0, L)$ are reduced to match the original range of indices before computing RoPE. In other words, the maximum relative distance between any two tokens has been reduced from L' to L and PI reduces the effect on attention score computation when extending the context window, and makes the LLMs easier to adapt. Furthermore, we define the scale parameter g as $\frac{L'}{L}$. For example, g is set as 2 when $L' = 8192$ for LLaMa2 with context window of $L = 4096$. Thus, the Eq. 3 can be reformulated as follows:

$$\mathbf{f}'(\mathbf{x}, m) = \mathbf{f}\left(\mathbf{x}, \frac{m}{g}\right). \tag{4}$$

Meanwhile, for PI, we need to collect the long-context data with the maximum length of L' in finetuning, and finetuning is needed for each extended window with high GPU memory usage as shown in Fig. 1 (a).

4 Method

In this section, we introduce the details of our E^2-LLM in Fig. 1 (b) for extending different sizes of context windows by only performing one training procedure on short-length data, which reduces the tuning costs greatly. First, in Sec. 4.1, we provide the necessary notations. Then, in Sec. 4.2.1, we illustrate the details of our E^2-LLM strategy to improve the length extension performance by introducing the augmentation on the scale, and the position offset parameters of RoPE, where these parameters are defined in Eq. 5. Finally, in Sec. 4.2.3, we show the training and inference processes in our E^2-LLM.

4.1 Notations

Apart from the notations defined in Sec. 3, we also define the following notations. First, the trained length is defined as R. It should mentioned that R is the maximum length of the data in finetuning, which is set as 8k in E^2-LLM, by default. Therefore, it is easy to collect the training data with a length of R and the used GPU memory in finetuning is also acceptable. In contrast, the trained length R is equal to the extension length L' (*e.g.*, 16k/32k) in many long-context extension methods (*e.g.*, PI), which requires high GPU memory usage in training. Second, we also introduce the position offset t in RoPE, and we can reformulate Eq. 4 to compute the RoPE embeddings as follows:

$$\mathbf{f}'(\mathbf{x}, m) = \mathbf{f}\left(\mathbf{x}, \frac{m+t}{g}\right). \tag{5}$$

In standard RoPE, by default, the t is set as 0. In our E^2-LLM, the t is selected from a range of indices $T = \{0, ..., t_{max}\}$, where t_{max} is the maximum position offset. Third, we also define a set of scale parameters used in E^2-LLM as $G = \{1, 2, ..., g_{max}\}$, where g_{max} is the maximum scale parameter.

4.2 E^2-LLM

In this section, we describe our proposed E^2-LLM strategy in detail. We take the LLM model $\mathcal{H}$ with the default context window L as 4,096 and the trained length R as 4,096 for illustration. We propose two different augmentation methods on the hyperparameters (*i.e.*, the scale parameter g and the position offset t) of RoPE.

4.2.1 Augmentation on g

As shown in Fig. 2, we illustrate the augmentation procedure on the scale parameter g.

In the training process of our proposed E^2-LLM, to make the model $\mathcal{H}$ cover different position densities in training, for the i-th iteration, we sample the scale parameter g_i from G for different iterations following a predefined probability distribution P as follows:

$$g_i = \mathcal{S}_g(P, G), g_i \in G, \tag{6}$$

where $\mathcal{S}_g(P, G)$ denotes the sampling operation on g, which samples g_i from set G following the distribution P. Therefore, different scale parameters are used for different iterations. Note that P is based on uniform distribution, by default.

In Fig. 2, we set the position offset t as 0, and then randomly select the scale parameter g from G for each sample based on Eq. 5, where g is set as 2, 5, and 10, respectively. Besides, as shown in Fig. 2, we observe that the interpolated maximum context windows are different for different samples in training, and the densities of the trained position indices are different. For example, the interpolated interpolated context windows are 8,192 and 20,480 when g is 2 and 5, respectively. Furthermore, as the training context window R is less than the interpolated maximum context windows, only a certain proportion of the position indices are trained, which are shown in blue in Fig. 2.

4.2.2 Augmentation on t

As shown in Fig. 2, we observe that we can only focus on a small range of the position indices when we start from zero index (i.e., $t = 0$). Therefore, to improve the robustness and generalization ability of our E^2-LLM, we further introduce the augmentation procedure on the position offset t by changing the absolute position indices of RoPE. Besides, inspired by several recent works (Han et al., 2023; Xiao et al., 2023), which claim that a large amount of attention scores are allocated to the initial tokens (i.e., attention sinks (Xiao et al., 2023)), we also keep several initial tokens and set the position offsets of these tokens as 0. For other position indices, in the i-th training iteration, we set the position offset t for different position indices of the trained window as follows:

$$t_i = \begin{cases} 0, & m \in [0, 3] \\ \mathcal{S}_t(Q, T), & m \in (3, R) \end{cases}, \tag{7}$$

where $\mathcal{S}_t(Q, T)$ denotes the sampling operation on t, and samples t_i from set T following the predefined probability distribution Q. Note that Q is

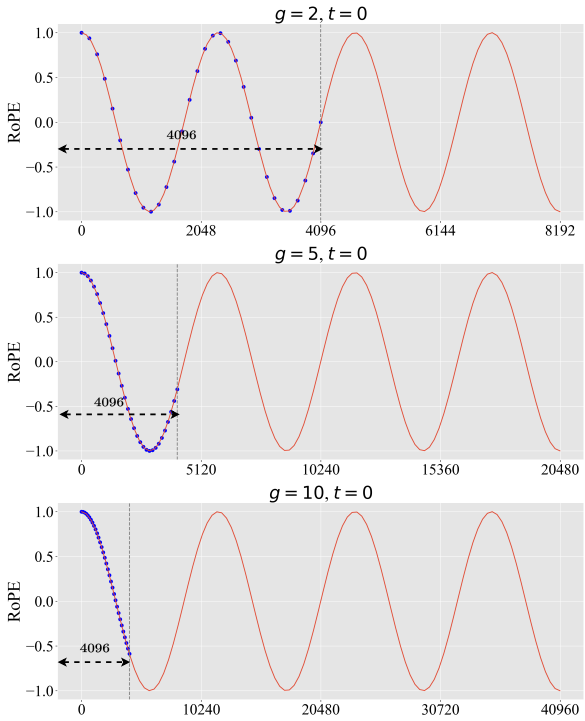

Figure 2: The trained position indices (blue points) when using different scale parameters (i.e., $g = 2, 5, 10$). The maximum length of the finetuning data (i.e., R) is 4096 and the position offset t is set as 0 for illustration.

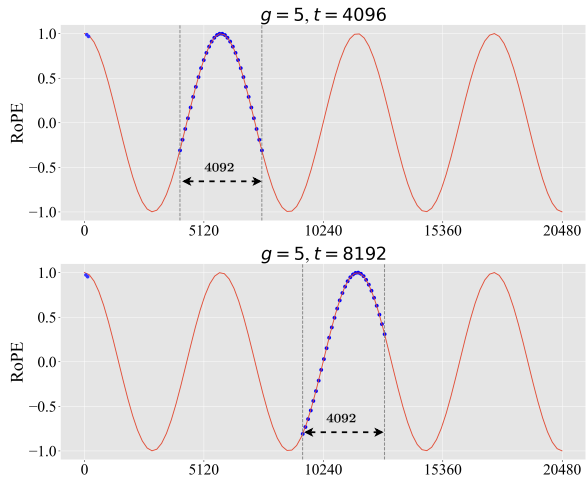

Figure 3: The trained position indices (blue points) when using different position offsets and g is set as 5 for visualization. The position indices of the first four tokens are not moved.

set as a uniform distribution and t_{max} is set as the difference between the maximum interpolated context window and the trained context window in the current iteration. Based on Eq. 7, for $n \in [0, 3]$ and $m \in (3, R)$, the Eq. 2 can be written as follows:

$$\begin{aligned} a(m,n) &= \mathrm{Re}\langle \mathbf{f}(\mathbf{q}, m+t_i), \mathbf{f}(\mathbf{k}, n+t_i)\rangle \\ &=: a(m + \mathcal{S}_t(Q,T) - n). \end{aligned} \tag{8}$$

Therefore, when $\mathcal{S}_t(Q,T)$ is larger, the range of relative position differences (i.e., $(m+\mathcal{S}_t(Q,T)-n)$) between m and n is larger, which will make the model generalize to different ranges of relative position differences.

In Fig. 3, we also provide the trained position indices (i.e., blue points) when introducing the augmentation on the position offset t, and observe that E^2-LLM can easily make use of the position indices with different absolute values and different ranges of relative differences.

4.2.3 Training and Inference

As discussed in Sec. 1, the training procedure is performed once for our E^2-LLM, and we can extend to different evaluation context windows easily at inference. The details are as follows.

Training. In training, first, for the i-th iteration, based on g_i and position offset t_i in training, we replace the g and t with g_i and t_i for Eg. 5, respectively. Then, we fine-tune the LLM $\mathcal{H}$ with a short context window R using the next token prediction task with modified position encodings on the trained context window. For better clarification, we also provide an algorithm of our proposed E^2-LLM method in Alg. 1.

Inference. Our E^2-LLM also does not introduce extra training weights, or modify the network architecture in any way, which means that it is attractive in practical applications as most infrastructure and optimization for the original model can be reused after length extension. At inference, we can extend to different context windows by setting different scale parameters for interpolation easily. For example, we set $g = 8$ for interpolating to $32,768$ and $g = 16$ for interpolating to $65,536$, which are called as E^2-LLM-32k and E^2-LLM-64k, respectively. It should be mentioned that the weights of E^2-LLM-32k and E^2-LLM-64k are the same at inference, and the only difference is that the scale parameters are set as 8 and 16, respectively. Moreover, in practice, we can only deploy one LLM on devices, and automatically change the scale parameter of RoPE based on the length of input context to support different context windows.

Table 1: Results (%) on single-doc QA, multi-doc QA and summarization tasks from LongBench dataset.

Model	Single-Doc QA				Multi-Doc QA				Summarization	
	Narrative QA	Qasper	MultiField QA-en	MultiField QA-zh	Hotpot QA	2WikiMulti hopQA	MuSi Que	Du Reader	Gov Report	QMSum
GPT-3.5-Turbo-16k	23.6	43.3	52.3	61.2	51.6	37.7	26.9	28.7	29.5	23.4
Llama2-7B-chat-4k	18.7	19.2	36.8	11.9	25.4	32.8	9.4	5.2	27.3	20.8
LongChat-v1.5-7B-32k	16.9	27.7	41.4	29.1	31.5	20.6	9.7	19.5	30.8	22.7
Vicuna-v1.5-7B-16k	19.4	26.1	38.5	43.0	25.3	20.8	9.8	19.3	27.9	22.8
LongLora-7B-16k	19.8	29.1	37.2	8.5	37.0	30.3	17.1	15.3	31.5	24.1
E^2-LLM-Llama2-7B-16k	16.4	34.7	39.1	43.6	37.1	34.4	17.9	18.6	29.4	23.0
E^2-LLM-Llama2-7B-32k	12.3	35.6	40.4	46.6	43.7	34.8	22.0	22.6	29.7	23.8
Llama2-13B-chat-4k	19.2	25.8	36.9	33.3	36.1	32.4	14.5	26.8	26.6	20.2
Vicuna-v1.5-13B-16k	18.9	29.9	46.2	28.4	38.1	36.0	10.7	20.9	27.9	22.1
PI-Llama2-13B-16k	19.2	33.3	42.7	47.9	44.9	34.8	19.5	17.4	27.9	23.7
E^2-LLM-Llama2-13B-16k	25.4	35.3	46.5	49.1	46.4	38.3	25.2	19.3	29.9	22.7
E^2-LLM-Llama2-13B-32k	24.1	36.2	49.0	52.5	49.2	37.6	23.1	20.4	29.9	23.1

Table 2: Results (%) on summarization, few-shot learning, synthetic, and code tasks from LongBench dataset. 'Overall' is computed by the macro-average over major task categories. This is computed on English (EN) tasks, Chinese (ZH) tasks, and all (All) tasks, code tasks are included in both languages.

Model	Summarization		Few-shot Learning				Code		Overall		
	MultiNews	VCSUM	TREC	TriviaQA	SAMSum	LSHT	LCC	RepoBench-P	EN	ZH	All
GPT-3.5-Turbo-16k	26.7	16.0	68.0	91.4	41.7	29.2	54.7	53.6	44.60	33.78	42.19
Llama2-7B-chat-4k	25.8	0.2	61.5	77.8	40.7	19.8	52.4	43.8	35.17	15.45	20.79
LongChat-v1.5-7B-32k	26.4	9.9	63.5	82.3	34.2	23.2	53.0	55.3	36.86	20.43	33.21
Vicuna-v1.5-7B-16k	27.2	15.1	74.0	86.2	40.8	28.8	51.0	43.5	36.49	26.55	34.28
LongLora-7B-16k	27.7	0.5	63.5	85.7	41.9	26.0	57.6	54.5	39.79	14.55	34.18
E^2-LLM-Llama2-7B-16k	25.9	9.6	68.5	89.2	38.2	35.0	65.8	58.1	41.26	26.70	38.03
E^2-LLM-Llama2-7B-32k	25.4	11.7	70.5	88.4	32.5	40.0	64.5	60.9	**41.74**	**30.23**	**39.18**
Llama2-13B-chat-4k	26.1	17.2	66.0	85.2	36.5	20.3	51.9	52.8	37.87	24.38	34.87
Vicuna-v1.5-13B-16k	27.1	16.4	74.0	84.9	27.8	29.8	44.1	45.6	38.08	23.86	34.92
PI-Llama2-13B-16k	25.9	9.1	72.5	86.5	27.9	31.0	62.5	51.1	40.88	26.35	37.65
E^2-LLM-Llama2-13B-16k	27.0	9.8	73.5	87.9	40.6	36.0	65.4	59.1	**44.73**	28.56	41.13
E^2-LLM-Llama2-13B-32k	26.8	10.2	75.0	87.8	40.9	44.5	63.8	57.5	44.55	**31.93**	**41.74**

5 Experiments

5.1 Experimental Settings

Models. In our E^2-LLM, we take the pre-trained 7B, 13B Llama2 (Touvron et al., 2023b) models to demonstrate the effectiveness of our E^2-LLM.

Training Procedure. All models are fine-tuned via the next token prediction objective based on two 8× A100 GPU machines. We use AdamW (Loshchilov and Hutter, 2019) with $\beta_1 = 0.9$ and $\beta_2 = 0.95$. The learning rate is set to 1×10^{-5} for 7B and 13B models, and the whole training step is set to 30,000 with a global batch size of 16.

Datasets. The training dataset includes the pre-train dataset (i.e., Pile (Gao et al., 2020)), and fine-tuning datasets (i.e., ShareGPT (Zheng et al., 2023) and the long summarization datasets (Cohan et al., 2018)). Note that the fine-tuning datasets are used to improve the question-answer abilities of long-context LLMs following Vicuna and LongChat models (Zheng et al., 2023) and generate reasonable results on LongBench. We evaluate the long-sequence language modeling performance of our fine-tuned models on the LongBench (Bai et al., 2023) and the arxiv proof-pile dataset (Azerbayev et al., 2022).

5.2 Results on LongBench

We evaluate several popular LLMs with long context capability, including GPT-3.5-Turbo-16k (OpenAI, 2022), Llama2-7B-chat-4k (Touvron et al., 2023b), LongChat-v1.5-7B-32k (Li et al., 2023), Vicuna-v1.5-7B-16k (Zheng et al., 2023), Longlora-7B-16k (Chen et al., 2023b), Llama2-13B-chat-4k (Touvron et al., 2023b), Vicuna-v1.5-13B-16k (Zheng et al., 2023), PI-Llama2-13B-16k. LongChat-v1.5-7B-32k, Vicuna-v1.5-7B-16k, and LongLora-7B-16k are fine-tuned from Llama2-7B

Algorithm 1 Training of E^2-LLM

Input: Pre-trained LLM model $\mathcal{H}$ with default context window of L (*e.g.*, 4k); The trained context window is R (*e.g.*, 4k/8k); The evaluation context window L' (*e.g.*, 32k/64k);

1: **for** the i-th iteration in training **do**
2: Set the scale g_i based on Eq. 6;
3: Set the position offset t_i based on Eq. 7;
4: Modify the RoPE position embeddings based on Eq. 5;
5: Train model $\mathcal{H}$ on training window R;
6: Compute the next token prediction loss;
7: Update parameters of model $\mathcal{H}$;
8: **end for**

Output: The optimized long context model $\mathcal{H}'$. (Note that $\mathcal{H}'$ can extend to different context windows at inference.);

based on PI. Vicuna-v1.5-13B-16k (Zheng et al., 2023), PI-Llama2-13B-16k are fine-tuned with Llama2-13B based on PI, where PI-Llama2-13B-16k are fine-tuned with our constructed datasets. Following LongBench (Bai et al., 2023), we conduct the assessment in a zero-shot setting, except for the few-shot learning tasks where the few-shot examples are provided as part of the long context. When the input length I surpasses the maximum context length L' of a model (indicated by the suffix of its name), we truncate the input sequence S from the middle since the front and end of the sequence may contain crucial information such as the instruction or question: $S_{1:I} \rightarrow [S_{1:\lfloor L'/2 \rfloor}; S_{I-\lfloor L'/2 \rfloor-1:I}]$. The metric for each dataset is shown in Table 6 from the Appendix A.1.

As shown in Table 1 and Table 2, we report the performance results (%) on the LongBench dataset. Specifically, the key findings from the experiment results are as follows: (1) When compared with the commercial model (GPT-3.5-Turbo-16k) with an overall accuracy of 44.60% in English, our E^2-LLM-Llama2-13B-32k achieves closing results with an overall accuracy of 44.55%. (2) In Table 1 and Table 2, we also evaluate the results of our E^2-LLM with different evaluation context sizes (i.e., 16k and 32k), and we observe that the performance results are better when we extend the evaluation context window size. Besides, as the lengths of most documents in LongBench are less than 16k, the improvements on these tasks are not significant when we increase the evaluation context window. (3) For a fair comparison, we also reimplement the positional interpolation method based on Llama2-13B (i.e., PI-Llama2-13B-16k) with the same training strategy and training datasets. When compared with PI-Llama2-13B-16k, our E^2-LLM-Llama2-13B-16k still achieves significant improvements on LongBench, which further demonstrates the effectiveness of our E^2-LLM.

5.3 Results on Proof-Pile

On the cleaned Arxiv Math proof-pile dataset (Azerbayev et al., 2022), we evaluate the long sequence language modeling performance of our extended models and baseline methods (i.e., Vicuna-v1.5-16k and LongChat-v1.5-32k), where the perplexity results on reported. For the proof-pile dataset, we randomly sample 128 documents with at least 64k tokens and evaluate the calculated perplexity of each of these samples. All perplexity evaluations were calculated using the sliding window method from (Press et al., 2022) with $S = 256$. Specifically, Vicuna-v1.5-16k and LongChat-v1.5-32k are fine-tuned on the Llama2 model and linear RoPE scaling method, which is based on the Position Interpolation (i.e., PI) (Chen et al., 2023a). In Table 3, we found that models extended with our method enjoy a significantly improved perplexity from longer context window sizes when compared with other baseline methods. Besides, for other methods, the training context window is equal to the maximum evaluation context window, thus the training costs are very large when the window size is large, where the training costs are shown in Fig. 1. In contrast, our E^2-LLM only needs to train Llama models once and the training context window is short, which reduces the training costs greatly.

5.4 Ablation Study

Effect of the augmentation strategies. In Table 4 we provide two alternative variants of our E^2-LLM (i.e., E^2-LLM (w/o aug on t), E^2-LLM (w/o aug on g)) to train the LLama2-13B base model. Specifically, for E^2-LLM (w/o aug on t), we only use the augmentation on the scale parameter g without using the augmentation on the position offset t, and we evaluate the performance by extending the context window as 32k. For E^2-LLM (w/o aug on g), we only use the augmentation on the position offset t and fix the scale parameter g as 2 with the training context window of 8k in our E^2-LLM. Note that the evaluation context window of E^2-LLM (w/o aug

Table 3: Evaluation perplexity on Arxiv Proof-pile dataset (Azerbayev et al., 2022) based on Llama2 7B and 13B models, where lower perplexity means better performance. "PI" denotes Position Interpolation (Chen et al., 2023a). The open-sourced Vicuna-v1.5-16k and LongChat-v.15-32k are extended based on the PI method. Note that the weights of E^2-LLM-16k, E^2-LLM-32k and E^2-LLM-64k are the same at inference, and the only difference is that the scale parameters are set as 4, 8 and 16, respectively.

Model			Evaluation Context Window Size				
Size	Training Context Window	Method	4096	8192	16384	32768	65536
7B	4k	None	2.92	-	-	-	-
7B	16k	Vicuna-v1.5-16k (PI)	3.48	3.17	3.95	-	-
	32k	LongChat-v1.5-32k (PI)	3.54	3.18	2.91	2.73	-
7B	4k	E^2-LLM-16k	2.96	2.71	2.54	-	-
		E^2-LLM-32k	2.99	2.74	2.56	2.46	-
		E^2-LLM-64k	3.06	2.81	2.62	2.51	2.56
13B	4k	None	2.78	-	-	-	-
13B	16k	Vicuna-v1.5-16k (PI)	3.27	2.97	2.78	-	-
13B	4k	E^2-LLM-16k	2.82	2.59	2.43	-	-
		E^2-LLM-32k	2.85	2.61	2.44	2.34	-
		E^2-LLM-64k	2.91	2.67	2.49	2.39	2.44

on g) is also set as 8k. As shown in Table 4, our E^2-LLM is better than these two alternative variants on LongBench, which shows that it is beneficial to perform the augmentation on t and g.

Table 4: Ablation on different augmentation strategies.

Methods	EN	ZH	All
E^2-LLM	**44.55**	**31.93**	**41.74**
E^2-LLM (w/o aug on t)	42.28	29.49	39.44
E^2-LLM (w/o aug on g)	41.66	28.33	38.77

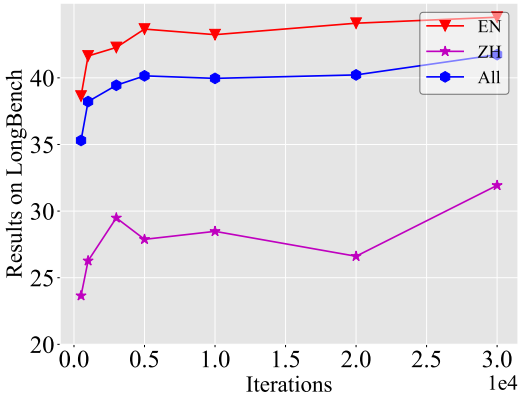

Figure 4: Performance results on the Longbench dataset when increasing the training steps.

Effect of the number of finetuning steps. As shown in Fig 4, we report the relationship between the results on the LongBench dataset and the number of fine-tuning steps for the LLaMA2 13B model using E^2-LLM, where the results are reported on the evaluation window size of 32k. In Fig. 4, at the first 5k iterations, the results improve quickly, which indicates that the models can gain long-context understanding abilities without a long training process. Furthermore, when increasing the training iterations, we observe that the performance results of LongBench can still increase steadily after 5k iterations.

Effect of the maximum scale parameter G_{max}. In Table 5, on LongBench dataset, we also report the results of our E^2-LLM based on the Llama2-13B model to analyze the effect of the maximum scale parameter G_{max} in training, and the evaluation context window is set as 32k. When G_{max} increases from 5 to 20, the performance results on Longbench are better. which indicates that it is effective to cover different densities by using a large G_{max} in our E^2-LLM. However, when we continue to increase the maximum scale parameter G_{max}, the performance improvement becomes relatively stable on LongBench. Thus, we directly set G_{max} as 20 to support a maximum extension window of 80k.

Table 5: Ablation on the maximum scale parameter G_{max}.

G_{max}	5	10	20	30
EN	43.20	44.25	44.55	44.01
ZH	29.33	30.28	39.93	32.76
All	40.12	41.15	41.74	41.51

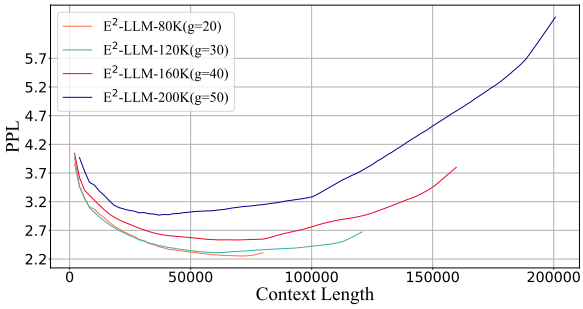

Figure 5: Generalization abilities on the unseen scales.

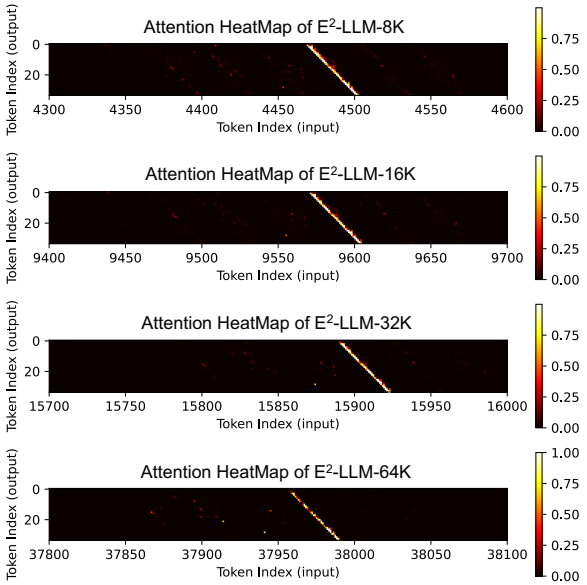

Figure 6: Visualization of attention heatmaps on 8k, 16k, 32k and 64k input contexts.

5.5 Further Analysis

Extension to unseen scales. By default, we set G_{max} as 20 to support the maximum interpolated context window of 80K. In Fig. 5, the interpolation scales are experimentally adjusted to 20, 30, 40, and 50 during inference to evaluate the generalization ability of E^2-LLM. The results demonstrate that PPL maintains a satisfactory level for contexts comprising fewer than 120K tokens. Nonetheless, when we continue to increase the scale, a discernible deterioration in performance occurs. It suggests that E^2-LLM possesses robust generalization ability for unseen or OOD scales within a certain range.

Visualization on Attention Map. To further analyze the effect of E^2-LLM, we visualize the attention heatmaps in the layer for our E^2-LLM-8k, E^2-LLM-16k, E^2-LLM-32k and E^2-LLM-64k based on Llama2-13B in Fig. 6 with the evaluation context windows of 8k, 16k, 32k and 64k by setting scale parameter as 2, 4, 8, 16, respectively. Specifically, as shown in Fig. 6, the vertical coordinate represents the indices of the generated sequence's token, and the horizontal coordinate represents the indices of the input sequence's tokens. The input text is part of a long paper, which is truncated to 16k, 32k and 64k, respectively. Then, three random key-value pairs are inserted into the input text, and a question is appended at the end of the text. Note the random key-value pairs and question are as shown in Appendix **??**, The question was answered correctly for 8k, 16k, 32k and 64k. In Fig. 6, we visualize the attention heatmaps of the output sequence corresponding to the input. The ground-truth indices of the values corresponding to the keys asked in 8k, 16k, 32k and 64k are [4470, 4503], [9572,9605], [15891,15924] and [37958, 37991], respectively, and we observe that the attention values of the output sequence at these positions are very significant, which represents that E^2-LLM can well index the correct position when generating the responses.

6 Conclusion

In this study, we introduce an Efficient and Extreme length extension method for LLMs, named E^2-LLM, which is designed to extend the context windows with a single training phase and minimal computational overhead. Notably, in our E^2-LLM, there is no requirement to amass extensive long-context datasets (*e.g.*, samples with 32k or 64k tokens) for training. Specifically, our E^2-LLM harnesses RoPE-based position embeddings to implement a pair of novel augmentation strategies that adjust the scale and position index parameters across different training samples with short lengths (*e.g.*, 4k/8k). Comprehensive experimental results on multiple benchmark datasets demonstrate the effectiveness of our E^2-LLM on long-context tasks.

7 Future Works

For the future directions, we have three plans as follows: (1) as our E^2-LLM is efficient and ef-

fective and we will try to apply our E^2-LLM on larger models (*e.g.*, LLama2 70B) and larger context windows (*e.g.*, 128k/192k); (2) we believe that our E^2-LLM is a general method and we will try to apply our E^2-LLM on more types of position encodings and more types of LLMs; (3) we will release our models and codes.

References

2019. Winogrande: An adversarial winograd schema challenge at scale.

2023. Ntk-aware scaled rope.

Zhangir Azerbayev, Edward Ayers, and Bartosz Piotrowski. 2022. Proof-pile.

Ge Bai, Jie Liu, Xingyuan Bu, Yancheng He, Jiaheng Liu, Zhanhui Zhou, Zhuoran Lin, Wenbo Su, Tiezheng Ge, Bo Zheng, and Wanli Ouyang. 2024. Mt-bench-101: A fine-grained benchmark for evaluating large language models in multi-turn dialogues. *arXiv*.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2023. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *CoRR*, abs/2004.05150.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Thirty-Fourth AAAI Conference on Artificial Intelligence*.

Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. 2022. Recurrent memory transformer. In *NeurIPS*.

Linzheng Chai, Jian Yang, Tao Sun, Hongcheng Guo, Jiaheng Liu, Bing Wang, Xiannian Liang, Jiaqi Bai, Tongliang Li, Qiyao Peng, et al. 2024. xcot: Cross-lingual instruction tuning for cross-lingual chain-of-thought reasoning. *arXiv preprint arXiv:2401.07037*.

Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. 2023a. Extending context window of large language models via positional interpolation. *CoRR*, abs/2306.15595.

Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2023b. Longlora: Efficient fine-tuning of long-context large language models. *arXiv:2309.12307*.

Arman Cohan, Franck Dernoncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. 2018. A discourse-aware attention model for abstractive summarization of long documents. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 615–621, New Orleans, Louisiana. Association for Computational Linguistics.

Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *CoRR*, abs/2307.08691.

Jiayu Ding, Shuming Ma, Li Dong, Xingxing Zhang, Shaohan Huang, Wenhui Wang, Nanning Zheng, and Furu Wei. 2023. Longnet: Scaling transformers to 1, 000, 000, 000 tokens. *CoRR*, abs/2307.02486.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.

Hongcheng Guo, Jian Yang, Jiaheng Liu, Liqun Yang, Linzheng Chai, Jiaqi Bai, Junran Peng, Xiaorong Hu, Chao Chen, Dongfeng Zhang, et al. 2023. Owl: A large language model for it operations. *arXiv preprint arXiv:2309.09298*.

Chi Han, Qifan Wang, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. 2023. Lm-infinite: Simple on-the-fly length generalization for large language models. *CoRR*, abs/2308.16137.

Gautier Izacard, Patrick S. H. Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2022. Few-shot learning with retrieval augmented language models. *CoRR*, abs/2208.03299.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick S. H. Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *EMNLP*, pages 6769–6781.

Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. In *ICLR*.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Matthew Kelcey, Jacob Devlin, Kenton Lee, Kristina N. Toutanova, Llion Jones, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association of Computational Linguistics*.

Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. 2017. Race: Large-scale reading comprehension dataset from examinations. *arXiv preprint arXiv:1704.04683*.

Hector Levesque, Ernest Davis, and Leora Morgenstern. 2012. The winograd schema challenge. Thirteenth international conference on the principles of knowledge representation and reasoning.

Dacheng Li, Rulin Shao, Anze Xie, Ying Sheng, Lianmin Zheng, Joseph E. Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. 2023. How long can open-source llms truly promise on context length?

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *ICLR*.

Amirkeivan Mohtashami and Martin Jaggi. 2023. Landmark attention: Random-access infinite context length for transformers. *CoRR*, abs/2305.16300.

OpenAI. 2022. Introducing chatgpt.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023. Yarn: Efficient context window extension of large language models. *CoRR*, abs/2309.00071.

Ofir Press, Noah A. Smith, and Mike Lewis. 2022. Train short, test long: Attention with linear biases enables input length extrapolation. In *ICLR*.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *KDD*, pages 3505–3506. ACM.

Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. 2019. Socialiqa: Commonsense reasoning about social interactions. *CoRR*, abs/1904.09728.

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2021. Roformer: Enhanced transformer with rotary position embedding. *CoRR*, abs/2104.09864.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023a. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023b. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288.

Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. *CoRR*, abs/2006.04768.

Zekun Moore Wang, Zhongyuan Peng, Haoran Que, Jiaheng Liu, Wangchunshu Zhou, Yuhan Wu, Hongcheng Guo, Ruitong Gan, Zehao Ni, Man Zhang, Zhaoxiang Zhang, Wanli Ouyang, Ke Xu, Wenhu Chen, Jie Fu, and Junran Peng. 2023. Rolellm: Benchmarking, eliciting, and enhancing role-playing abilities of large language models. *arXiv preprint arXiv: 2310.00746*.

Yanan Wu, Jie Liu, Xingyuan Bu, Jiaheng Liu, Zhanhui Zhou, Yuanxing Zhang, Chenchen Zhang, Zhiqi Bai, Haibin Chen, Tiezheng Ge, Wanli Ouyang, Wenbo Su, and Bo Zheng. 2024. Conceptmath: A bilingual concept-wise benchmark for measuring mathematical reasoning of large language models. *arXiv*.

Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. 2022. Memorizing transformers. In *ICLR*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv*.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big bird: Transformers for longer sequences. In *NeurIPS*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena.

Input Context

Key-value pairs: Extract the value corresponding to the specified key in the JSON object below. {"2a8d601d-1d69-4e64-9f90-8ad825a74195": "bb3ba2a5-7de8-434b-a86e-a88bb9fa7289", "9f4a92b9-5f69-4725-ba1e-403f08dea695": "703a7ce5-f17f-4e6d-b895-5836ba5ec71c", "52a9c80c-da51-4fc9-bf70-4a4901bc2ac3": "b2f8ea3d-4b1b-49e0-a141-b9823991ebeb"}
Question: What is the value of key "9f4a92b9-5f69-4725-ba1e-403f08dea695"?

A More Details

A.1 More details of the LongBench dataset

The details are shown in Table 6.

A.2 More details of the attention map visualization

The three random key-value pairs and the input question are shown as follows.

Table 6: An overview of the LongBench dataset. "Source" denotes the origin of the context. "Avg len" (average length) represents the mean length, which is calculated by counting words in English (code) datasets and characters in Chinese datasets. "Accuracy (CLS)" and "Accuracy (EM)" are classification accuracy and exact match accuracy, respectively.

Dataset	ID	Source	Avg len	Metric	Language	#data
Single-Document QA						
NarrativeQA	1-1	Literature, Film	18,409	F1	English	200
Qasper	1-2	Science	3,619	F1	English	200
MultiFieldQA-en	1-3	Multi-field	4,559	F1	English	150
MultiFieldQA-zh	1-4	Multi-field	6,701	F1	Chinese	200
Multi-Document QA						
HotpotQA	2-1	Wikipedia	9,151	F1	English	200
2WikiMultihopQA	2-2	Wikipedia	4,887	F1	English	200
MuSiQue	2-3	Wikipedia	11,214	F1	English	200
DuReader	2-4	Baidu Search	15,768	Rouge-L	Chinese	200
Summarization						
GovReport	3-1	Government report	8,734	Rouge-L	English	200
QMSum	3-2	Meeting	10,614	Rouge-L	English	200
MultiNews	3-3	News	2,113	Rouge-L	English	200
VCSUM	3-4	Meeting	15,380	Rouge-L	Chinese	200
Few-shot Learning						
TREC	4-1	Web question	5,177	Accuracy (CLS)	English	200
TriviaQA	4-2	Wikipedia, Web	8,209	F1	English	200
SAMSum	4-3	Dialogue	6,258	Rouge-L	English	200
LSHT	4-4	News	22,337	Accuracy (CLS)	Chinese	200
Code Completion						
LCC	6-1	Github	1,235	Edit Sim	Python/C#/Java	500
RepoBench-P	6-2	Github repository	4,206	Edit Sim	Python/Java	500

Table 7: Performance on a subset of general benchmarks.

Model	PIQA	WSC	HellaSwag	SIQA	WinoGrande	Race-H	Race-M	NaturalQuestions	**Avg.**
Llama2-7B (4K)	78.1	67.3	73.0	48.1	69.5	40.2	37.1	16.4	53.7
E^2-LLM-32K	77.6	66.4	71.9	45.9	69.5	47.6	40.9	18.8	54.8
E^2-LLM-64K	77.2	67.3	70.9	46.5	69.4	44.4	38.8	17.5	54.0
Llama2-13B (4K)	78.9	64.4	75.7	51.7	73.5	63.0	58.9	20.2	60.8
E^2-LLM-32K	79.3	67.3	75.6	61.3	74.4	67.3	59.8	26.8	64.0
E^2-LLM-64K	78.8	67.3	75.0	61.1	74.5	61.7	55.8	25.2	62.4

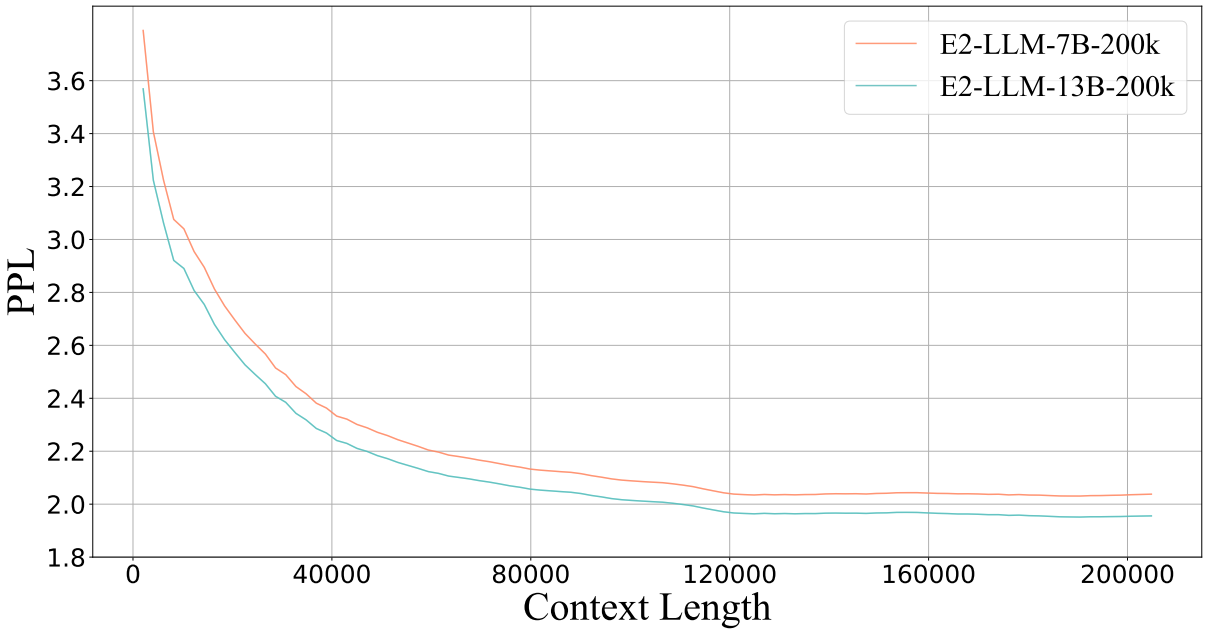

Figure 7: Results of E^2-LLM on 200K context window.

A.3 More results on short-length datasets

We evaluate the models extended by E^2-LLM on several standard short-length benchmark tasks (i.e., PIQA (Bisk et al., 2020), WSC (Levesque et al., 2012), HellaSwag (Zellers et al., 2019), SIQA (Sap et al., 2019), WinoGrande (ai2, 2019), RACE (Lai et al., 2017), and NaturalQuestions (Kwiatkowski et al., 2019)) within the original context window size of 4,096, where the zero-shot performance results are reported in Table 7. Specifically, we report the results of extended models (E^2-LLM-32K and E^2-LLM-64K) on Llama2-7B and Llama2-13B models, respectively. From the results of Table 7, when compared with the baseline Llama2 models, we observe that the extended Llama2 models (*e.g.*, E^2-LLM-32K, E^2-LLM-64K) still preserve comparable or even better performance results on these short-length benchmark datasets, which further demonstrates the effectiveness of E^2-LLM on the short-context interpretation.

A.4 Extension on our E^2-LLM

Recently, we have updated the methodology and training strategies of our E^2-LLM, and successfully extended the Llama2-7B and 13B to 200k context windows with dramatically lower training costs as shown in Fig. 7. In Fig. 7, we report the PPL results on several documents truncated to 200k from Proof-Pile, and we observe that the PPL results decrease smoothly, which further demonstrate the effectiveness of our E^2-LLM. We will add more implementation details on our newly updated E^2-LLM method in the future.